Http Module and Express.js

http Module: The HTTP module in Node.js is a ***built-in core module used to create and manage HTTP servers and clients, enabling handling of requests, responses, and headers directly in JavaScript.***

Key Features

- **Server Creation:** Use ***http.createServer*** () to create an HTTP server that listens on a specified port and executes a callback for each request.
- **Request Handling**: The server receives an ***IncomingMessage object (req)*** representing the client request, including **headers**, **URL**, and **method**.
- **Response Handling:** The server sends responses using the ***ServerResponse object (res), with methods like res.write()*** to send data and ***res.end()*** to signal completion.
- **HTTP Methods Support**: Supports standard methods such as **GET**, **POST**, **PATCH, PUT, DELETE.**
- **Header Management:** Allows reading request headers via req.headers and setting response headers with ***res.writeHead()***
- **Streaming**: Handles large or chunked messages efficiently without buffering the entire request or response

Expectation : To know the working of http Module

Create Server: **** Let's JUMP into VS Code ****





What is Express.js

Express.js is a **fast, unopinionated (flexibility), and minimalist web framework** for **Node.js** *that simplifies building server-side applications and APIs.*

Note: It does not replace the http module, but remove the repetitive work.

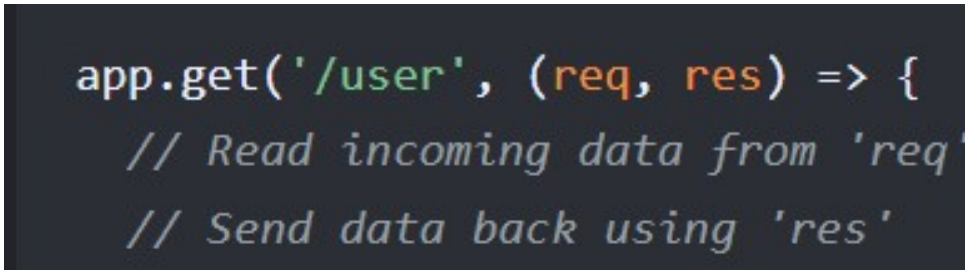
Acting as a thin layer over Node's HTTP module, it provides powerful tools for **routing**, **middleware handling**, and **HTTP utilities**, making backend development more organized and efficient.

Key capabilities include:

- **Minimal & Flexible**: Only core features by default, with the ability to add middleware as needed. **(Flexibility)**
- **Powerful Routing**: Clean handling of HTTP methods like GET, POST, PUT, and DELETE.
- **Middleware Support**: Easily integrate authentication, logging, error handling, and more.
- **Templating Engine Integration**: Supports Pug, EJS, Handlebars for dynamic HTML rendering.
- **Static File Serving**: Serve images, CSS, and JS directly.
- **REST API Friendly**: Ideal for CRUD and RESTful services.

Request (req) and Response (res)

In express.js **`req`** and **`res`** are objects passed as parameters to route handlers and middleware functions. They represent the **HTTP request-response cycle**: **`req`** stands for **Request** (what comes into the server) and `res` stands for **Response** (what goes back to the client.



The Request Object (req)

The req object contains all the information about the **incoming HTTP request** sent by the client (such as a browser, mobile app, or frontend framework).

You use req to extract data, check headers, or see who is making the request. Some of its most commonly used properties include:

- **req.body**: Holds data submitted in the request body (typically used in POST or PUT requests, like submitting a form or sending JSON).
- **req.params**: Contains routing parameters specified in the URL path (e.g., if the route is `/user/:id`, `req.params.id` grabs that specific ID).
- **req.query**: Contains the query string parameters from the URL (e.g., in **`/search?term=apple`**, **`req.query.term`** will be **`"apple"`**).
- **req.headers**: Contains the HTTP headers sent by the client, such as Authorization tokens or Content-Type.
- **req.method**: The HTTP method used (e.g., GET, POST, DELETE).

The Response Object (res)

The res object represents the **HTTP response** that the Express server sends back to the client.

You use res to deliver data (like HTML, text, or JSON), set status codes, or redirect the user. Some of its most commonly used methods include:

- **res.send()**: Sends a basic HTTP response. It can automatically set the content type depending on whether you pass a string, object, or array.
- **res.json()**: Sends a JSON response. This is heavily used when building REST APIs.
- **res.status()**: Sets the HTTP status code (e.g., 200 for success, 404 for not found, 500 for server error). It is usually chained with a response method like `res.status(404).send('Not Found')`.
- **res.redirect()**: Redirects the client's browser to a different URL path.
- **res.sendFile()**: Streams a file directly to the client (like an image or a static HTML page).

---------- **********----------

What is Dependencies and dev-dependencies

What are Middleware, Controller

Middleware is a function that executes during the HTTP request-response cycle.

- It sits directly between the incoming request (**req**) and the final route handler, giving you access to the **req** object, the **res** object, and the **next** function.

Middleware can end the req-res lifecycle OR can pass the control to next Route handler